



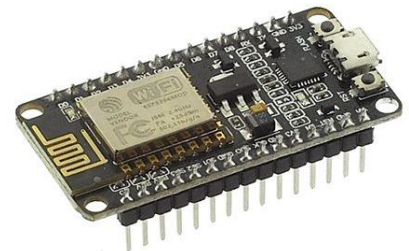
## Créer un serveur web avec un microcontrôleur ESP8266

### Objectifs :

- Etudier la programmation en langage Python pour microcontrôleur (MicroPython)
- Créer une connexion wifi entre une carte ESP et un réseau wifi local
- Créer un page web, consultable sur un périphérique mobile,
- Piloter une sortie booléenne à travers un réseau wifi.

### Objectif de l'étude

- Créer un système domotique à base de carte ESP8266 programmé en Python permettant de gérer l'éclairage d'une maison à partir d'une interface web connecté au wifi de l'habitation.

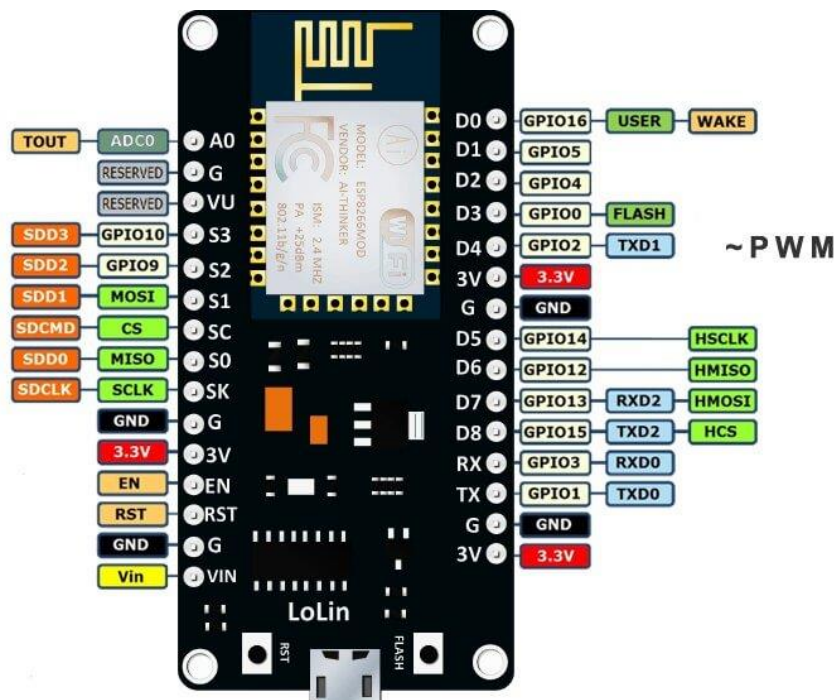


### Présentation de la carte ESP8266

Le NodeMCU ESP8266 est un microcontrôleur avec un module Wifi intégré. Très simple d'utilisation il est léger et possède une capacité de mémoire et de calcul supérieure aux Arduino.

Le microcontrôleur NodeMCU ESP8266 utilise le microprocesseur Tensilica 32-bit RISC CPU Xtensa LX106. Ce processeur fonctionne à une fréquence d'horloge de 80 MHz. Il possède une mémoire RAM de 64 kB, EEPROM et aussi une mémoire Flash de 4000 kB (pour la programmation et l'enregistrement de données).

Le microcontrôleur possède une puce Wifi permettant de se connecter au réseau local, de créer un serveur ou de créer son propre réseau afin que d'autre appareils s'y connectent. C'est cette dernière caractéristique qui va nous intéresser dans cette activité.



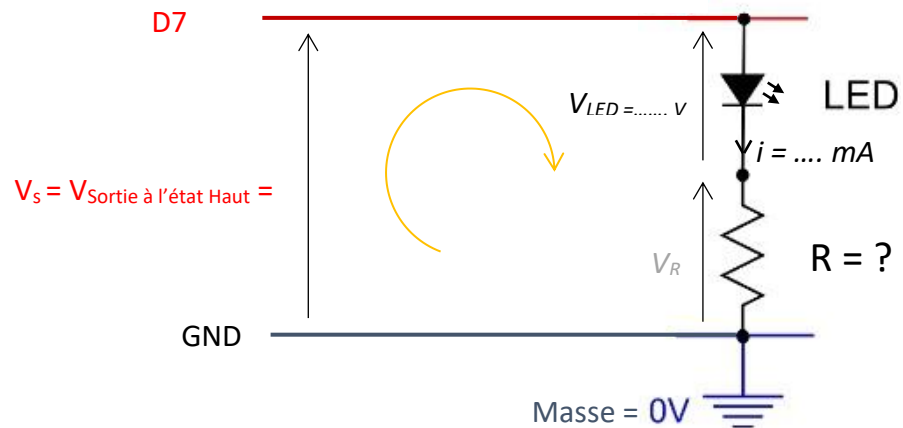


## Câblage du circuit

### Détermination de la résistance de protection

Afin de permettre l'allumage de la LED tout en la préservant, il est nécessaire de câbler une résistance en série dans son circuit d'alimentation.

Le schéma de câblage de la LED est le suivant :



Question 1. En établissant la loi des mailles dans le circuit, exprimer  $V_R$  en fonction de  $V_s$  et  $V_{LED}$ .

Question 2. En écrivant la loi d'Ohm aux bornes de la résistance, en déduire la relation entre  $R$ ,  $V_s$ ,  $V_{LED}$

La tension  $V_s$  et le courant  $i$  sont caractéristiques du microcontrôleur ESP8266 :

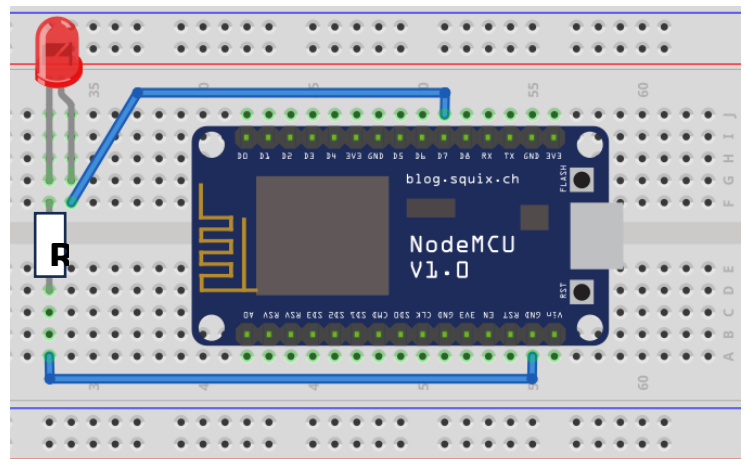
$V_s = 3.3 V$   $i = 20 mA$  (fourni par une sortie du ESP8266)

Question 3. A partir de la documentation technique de la LED fournie (Doc\_tech\_LED.pdf), déterminer la tension moyenne de fonctionnement de la LED (notée Forward Voltage)

Question 4. En déduire la valeur de la résistance.

### Réalisation du câblage

Activité 1 : Après avoir choisi une résistance adéquate, réaliser le câblage suivant (hors tension bien sûr...). Et le faire vérifier avant de mettre sous tension.





### Programmation de la carte

L'ESP8266 peut se programmer soit en C++ en utilisant l'IDE Arduino, soit en microPython. Le changement de support de programmation se fait en modifiant le firmware de la carte. La carte à votre disposition a été préparée pour être programmée en MicroPython et on utilisera le logiciel Thonny.

Il faut brancher votre carte en USB avant de lancer Thonny.

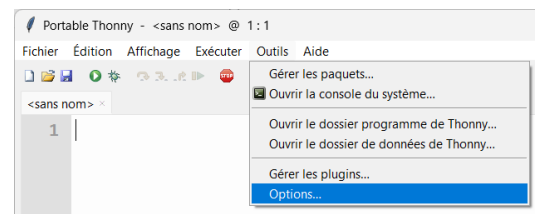
La mémoire de la carte contient 2 programmes :

- **boot.py** : qui est exécuté une seule fois lors du démarrage de la carte. Cela inclut l'importation de bibliothèques, d'informations d'identification réseau, l'instanciation des broches, la connexion à votre réseau et d'autres configurations.
- **main.py** : qui contient le programme à exécuter.

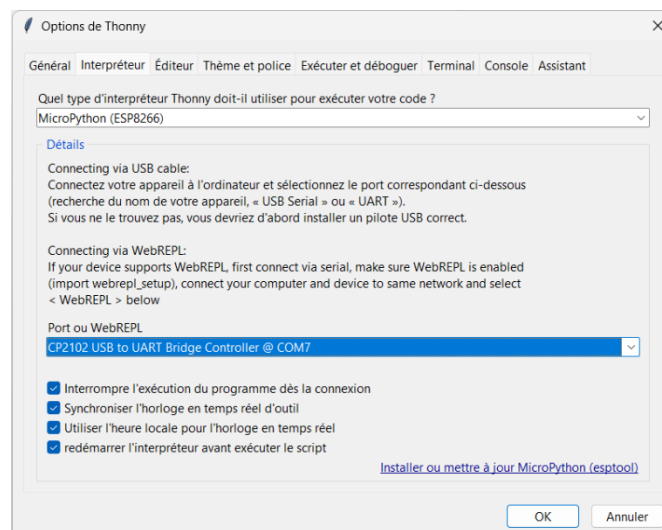
### Préparation de l'IDE Thonny

Thonny est un IDE Python classique et il faut le paramétrer pour qu'il puisse gérer plus facilement la carte ESP8266 connectée.

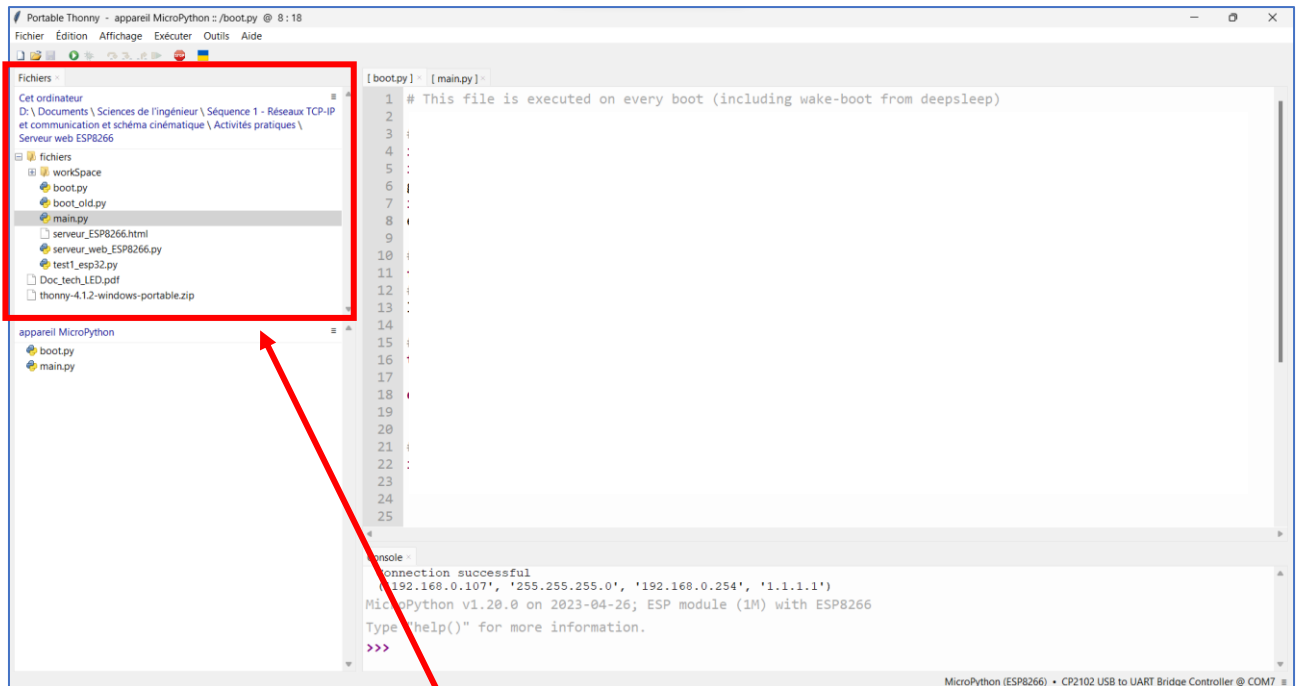
#### Activité 2 : Sélectionner Outils puis Options



#### Activité 3 : Paramétrer l'interpréteur MicroPython (ESP8266) pour exécuter le code. Il faut également sélectionner le port COM sur lequel est connectée votre carte.



En cliquant sur l'onglet **Affichage** puis sur **fichiers**, on peut avoir accès aux fichiers présents dans notre espace de travail et sur la carte.



## La programmation

Activité 4 : Dans l'explorateur de fichier de Thonny, ouvrir les fichiers **main\_eleve.py** et **boot\_eleve.py** présent dans votre espace de travail.

## Boot.py

Activité 5 : Analyser le script à l'aide des commentaires et de l'aide du professeur.

Activité 6 : Compléter la ligne 14 du programme avec le numéro de la broche (numérotation GPIO) sur laquelle est connectée l'anode de la LED.

```

9
10 #importation du module qui permet d'utiliser le GPIO de la carte
11 from machine import Pin
12 #A COMPLETER
13 #Déclaration de la PIN GPIO connectée à la LED comme une sortie
14 led = Pin(..., Pin.OUT)
15

```

Activité 7 : Compléter les lignes 27 et 29 avec les caractéristiques du réseau wifi disponible :

SSID (Service Set Identifier) : **speSINaragon**  
 Password : **aragonSIN2022!!**

Activité 8 : Sélectionner « enregistrer sous... » et sauvegarder votre programme sur la carte en le nommant **boot.py**



### Main.py

Remarque : Cette partie permet de comprendre la structure du fichier. Il est conseillé de bien prendre le temps d'analyser ce fichier afin de permettre sa modification dans la partie suivante.

Le fichier **main.py** permet entre autres, de stocker et d'afficher sur le client, la page web permettant de piloter le système.

#### - La fonction **web\_page()** :

Cette fonction python permet d'afficher la page web du serveur. Elle teste l'état de la LED (ON ou OFF) et le stocke dans une variable **gpio\_state** au format texte.



Cette variable est utilisée dans le fichier html de la page web.

```
<body>
<h1>ESP Serveur domotique</h1>
<q>Cliquer pour activer/désactiver l'éclairage</q>
<hr/>
<p>Eclairage du salon : <strong>"" + gpio_state + ""</strong></p>
<p><a href="/?led=on"><button class="button">ON</button></a></p>
<p><a href="/?led=off"><button class="button button2">OFF</button></a></p>
</body>
```

```
4 def web_page():
5     #la page html affichant l'état de la LED,
6     if led.value() == 1:
7         gpio_state = "ON"
8     else:
9         gpio_state = "OFF"
10
11     html = ""<html><head><title>Serveur domo
12     return html
```

#### - Création du socket de connexion :

Un socket ? une chaussette ??

Dans le fichier boot.py, nous avons demandé au serveur DHCP d'attribuer une adresse IP à notre carte ESP. Cependant, il est également nécessaire de définir un port de connexion. En effet, le port de connexion détermine l'application ou le service mise en œuvre dans notre serveur (l'ESP donc).

Ici, on souhaite héberger un site web et donc utiliser le protocole HTTP. Ce protocole est rattaché au port système 80.

Les applications fonctionnent généralement au-dessus de TCP ou d'UDP, et sont souvent associées à un *port bien connu*. Exemples :

- HTTP port TCP 80 ;
- SSH port TCP 22 ;
- DNS port UDP 53 (TCP 53 pour les transferts de zones et les requêtes supérieures à 512 octets) ;
- RIP port UDP 520 ;
- FTP port TCP 21 ;

wikipedia

Cela signifie que lorsque l'on se connecte avec navigateur web, il envoie deux informations : l'adresse IP du serveur et le numéro de protocole 80 correspondant au HTTP.

Il faut donc lier 2 données pour notre serveur, son adresse IP et le numéro de port 80.

En programmation, on utilise un objet appelé **SOCKET BSD** (Berkeley Sockets Interface, du nom de l'université des ingénieurs qui l'ont créé).

On résume : on crée donc une variable appelée serveur permettant de stocker un socket puis on attend une connexion sur le port 80, correspondant à un client qui consulte un site web. On met une limite à 5 tentatives de connexion pour éviter qu'une mauvaise connexion ne paralyse notre serveur (choix arbitraire et empirique)

```
14 serveur = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
15 serveur.bind(('', 80))
16 serveur.listen(5)
```



## - Boucle principale :

En Arduino c'est un **void loop()** :

en python c'est un **while True** :

Activité 9 : « Enregistrer sous... » le fichier main.py dans la carte (demander de l'aide si erreur, c'est capricieux). Puis appuyer sur le bouton Reset (RST) de la carte.

En examinant les messages dans la console, on peut voir les messages suivants :

On accepte la connexion et on stocke l'adresse IP du client (variable addr).

**Got a connection from ('172.18.X.X', XXXXX)**

Ici, on écrit dans la console python, l'IP du client et le port d'émission de la requête. Il peut être très variable car un navigateur crée un nouveau port à chaque nouvel onglet ouvert.

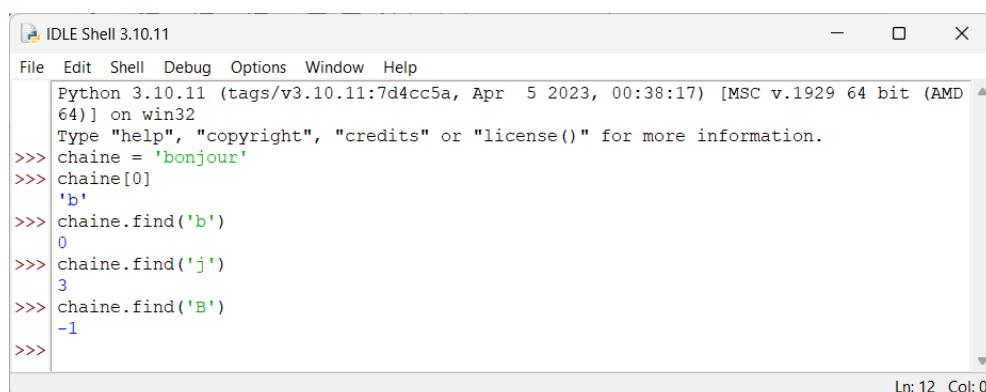
```
18 while True:
19     conn, addr = serveur.accept()
20     print('Got a connection from %s' % str(addr))
21     request = conn.recv(1024)
22     request = str(request)
23     print('Content = %s' % request)
24     led_on = request.find('/?led=on')
25     led_off = request.find('/?led=off')
26     if led_on == 6:
27         print('LED ON')
28         led.value(1)
29     if led_off == 6:
30         print('LED OFF')
31         led.value(0)
32     response = web_page()
33     conn.send('HTTP/1.1 200 OK\n')
34     conn.send('Content-Type: text/html\n')
35     conn.send('Connection: close\n\n')
36     conn.sendall(response)
37     conn.close()
```

La commande de la ligne 23 écrit dans la console la requête du client reçu à la ligne 21 et convertie en chaîne de caractère à la ligne 22.

Voici le texte de cette requête (exemple sur mon réseau personnel) :

```
b'GET /?led=off HTTP/1.1\r\nHost: 192.168.0.52\r\nConnection: keep-alive\r\nCache-Control:
max-age=0\r\nUpgrade-Insecure-Requests: 1\r\nUser-Agent: Mozilla/5.0 (Linux; Android 10; K
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/116.0.0.0 Mobile Safari/537.36\r\nAccept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;
q=0.8\r\nSec-GPC: 1\r\nAccept-Language: fr-FR,fr\r\nReferer:
http://192.168.0.52/?led1=off\r\nAccept-Encoding: gzip, deflate\r\n\r\n'
```

Activité 10 : Ouvrir une console Python (chercher IDLE dans le menu démarrer) et saisir les commandes suivantes :



```
IDLE Shell 3.10.11
File Edit Shell Debug Options Window Help
Python 3.10.11 (tags/v3.10.11:7d4cc5a, Apr 5 2023, 00:38:17) [MSC v.1929 64 bit (AMD
64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> chaine = 'bonjour'
>>> chaine[0]
'b'
>>> chaine.find('b')
0
>>> chaine.find('j')
3
>>> chaine.find('B')
-1
>>>
```

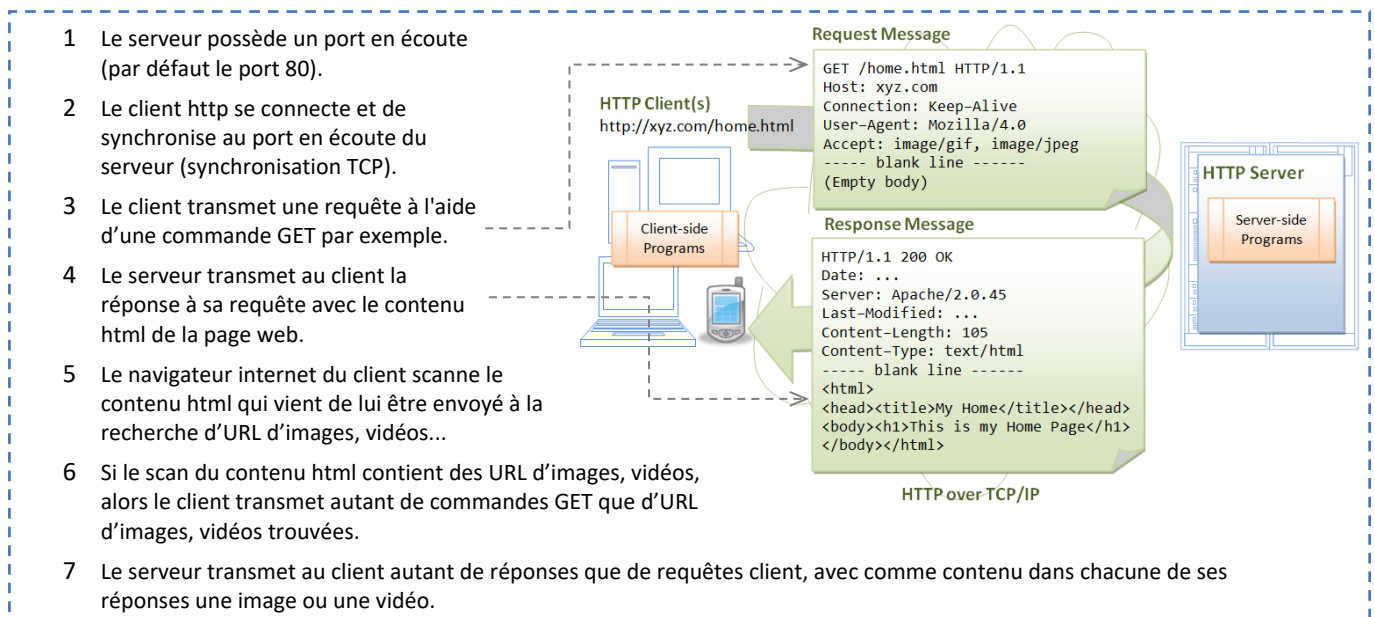


Question 1. D'après le comportement examiné dans la console Python, analyser la fonction de la méthode `.find()`.

Question 2. En déduire la valeur de test `'led_on == 6'` et `'led_off == 6'`.

## Le Protocole HTTP

Que se passe-t-il lors d'une requête client pour afficher une page web ?  
(ici la page web <http://xyz.com/home.html>).



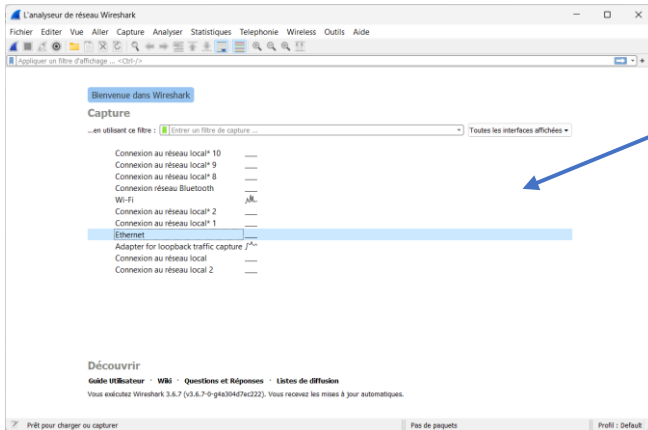
Dans le protocole http, une méthode est un type de requête, c'est-à-dire qu'elle demande au serveur d'effectuer une action. Les méthodes les plus utilisées sont GET et POST :

- **GET** : c'est la méthode la plus courante pour demander une ressource. Une requête GET est sans effet sur la ressource.
- **POST** : cette méthode doit être utilisée pour ajouter une nouvelle ressource, comme un message sur un forum, un article dans un site ou encore un login et un mot de passe.

### Visualisation de la requête GET client

Le programme stocké sur le serveur ne permet pas de visualiser la requête du client. Pour cela, il faut utiliser un scanner de trame : WIRESHARK.

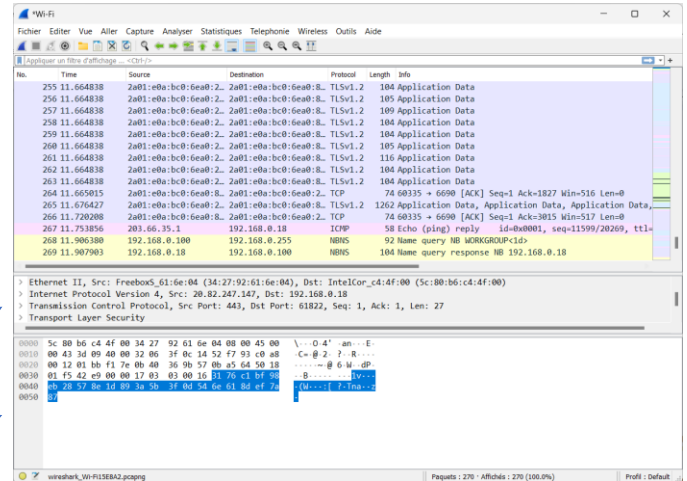
Activité 11 : Exécuter le programme **Wireshark** disponible sur le réseau (vous aurez besoin d'un login administrateur... demandez !) et suivre le tutorial suivant :



Choisir la carte réseau que vous souhaitez scanner. Pour vous : l'Ethernet.

3 écrans :

- Les trames du réseau dans l'ordre temporel
- L'analyse d'une trame suivant les couches OSI
- Le contenu numérique d'une trame représenté en hexadécimal.

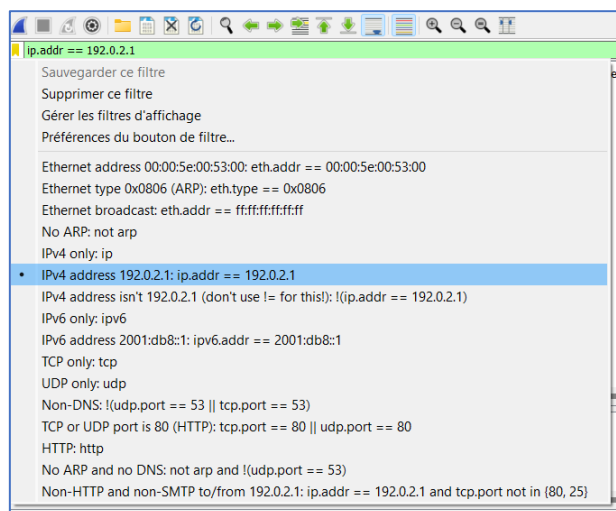


Activité 12 : Si besoin, relancer le programme main en appuyant à nouveau sur RST.

Activité 13 : Sur votre client (PC ou mobile), consulter la page web et appuyer sur les boutons ON et OFF.

Activité 14 : Sur Wireshark, arrêter le scan en cliquant sur .

Filtrer les trames dans la barre de recherche pour n'afficher que les trames concernées par votre serveur (voir IP sur la console Python)





Activité 15 : Chercher dans les trames affichées la requête client : GET .....  
Ainsi que la réponse du serveur.

Les ligne 33 à 35 programment cette réponse http du serveur à la requête du client.

Cette réponse fait suite à la requête du client qui demande l'affichage de la page web.

```
18 while True:
19     conn, addr = serveurur.accept()
20     print('Got a connection from %s' % str(addr))
21     request = conn.recv(1024)
22     request = str(request)
23     print('Content = %s' % request)
24     led_on = request.find('/?led=on')
25     led_off = request.find('/?led=off')
26     if led_on == 6:
27         print('LED ON')
28         led.value(1)
29     if led_off == 6:
30         print('LED OFF')
31         led.value(0)
32     response = web_page()
33     conn.send('HTTP/1.1 200 OK\n')
34     conn.send('Content-Type: text/html\n')
35     conn.send('Connection: close\n\n')
36     conn.sendall(response)
37     conn.close()
```

## Mini Projet : Ajouter un second circuit d'éclairage piloté.

L'objectif du travail demandé est de rajouter la gestion de l'éclairage d'une seconde pièce.

Pour cela, vous devrez :

- Câbler une LED modélisant la lampe de la cuisine,
- Modifier la page **serveur\_ESP8266.html** en fonction de la capture d'écran ci-contre,
- Modifier le fichier **boot.py** pour déclarer la nouvelle LED,
- Modifier le fichier **main.py** afin d'intégrer la nouvelle page web (demander de l'aide) et la gestion de la deuxième LED.

